

✓ HWO - Turn your favorite photo a rotating video

Total Points: 100

Assignment Due: Sep 11th Thursday 11:59 PM EST

In this assignment, you will:

- Implement rotation using **forward mapping** (origin & arbitrary center)
- Implement rotation using **inverse mapping** (origin & arbitrary center)
- Animate your image rotating and export as an MP4 video

Please submit the following files on Canvas:

- The original .ipynb notebook
- A PDF version of the .ipynb notebook
- rotation.mp4
- rotating_favorite_video.mp4
- rotating_favorite_creative_video.mp4

Allowed libraries: **numpy**, **matplotlib (plt)**, **imageio**, **opencv-python (cv2)**

```
1 !pip install numpy matplotlib imageio opencv-python
```

```
Requirement already satisfied: numpy in /Users/victorli/Documents/GitHub/CS442/.venv/lib/python3.13/site-packages (2.
Requirement already satisfied: matplotlib in /Users/victorli/Documents/GitHub/CS442/.venv/lib/python3.13/site-package
Requirement already satisfied: imageio in /Users/victorli/Documents/GitHub/CS442/.venv/lib/python3.13/site-packages (
Requirement already satisfied: opencv-python in /Users/victorli/Documents/GitHub/CS442/.venv/lib/python3.13/site-pack
Requirement already satisfied: contourpy>=1.0.1 in /Users/victorli/Documents/GitHub/CS442/.venv/lib/python3.13/site-p
Requirement already satisfied: cycler>=0.10 in /Users/victorli/Documents/GitHub/CS442/.venv/lib/python3.13/site-packa
Requirement already satisfied: fonttools>=4.28.2 in /Users/victorli/Documents/GitHub/CS442/.venv/lib/python3.13/site-
Requirement already satisfied: kiwisolver>=1.3.1 in /Users/victorli/Documents/GitHub/CS442/.venv/lib/python3.13/site-
Requirement already satisfied: packaging>=20.0 in /Users/victorli/Documents/GitHub/CS442/.venv/lib/python3.13/site-pa
Requirement already satisfied: pillow>=9 in /Users/victorli/Documents/GitHub/CS442/.venv/lib/python3.13/site-packages
Requirement already satisfied: pyparsing>=3 in /Users/victorli/Documents/GitHub/CS442/.venv/lib/python3.13/site-packa
Requirement already satisfied: python-dateutil>=2.7 in /Users/victorli/Documents/GitHub/CS442/.venv/lib/python3.13/si
Requirement already satisfied: six>=1.5 in /Users/victorli/Documents/GitHub/CS442/.venv/lib/python3.13/site-packages
```

[notice] A new release of pip is available: 24.2 -> 26.2.1

[notice] To update, run: `pip install --upgrade pip`

Use `imageio.imread` to load the sample image.

```
1 import imageio
2 import matplotlib.pyplot as plt
3
4 img = imageio.imread('Lenna.png')
5 plt.imshow(img)
6 plt.axis('off')
7 plt.show()
```

```
/var/folders/hb/3k_51w_j5yd8v5cpqy1brfqr0000gn/T/ipykernel_46027/3604052135.py:4: DeprecationWarning: Starting with I
img = imageio.imread('Lenna.png')
```



✓ Part A. Forward Mapping (30 points)

Explanation

- **Equation:**

$$x_d = R(\theta) x_s$$

where

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

- **Insight:** Where will the pixel land in the destination? (Throwing pixels forward).
- **Pseudocode:**

```
for each pixel (x_s, y_s) in source:
    [x_d, y_d] = R(theta) * [x_s, y_s]
    put source(x_s, y_s) into destination(round(x_d), round(y_d))
```

✓ Q1-1. Implement `forward_mapping_origin` (15 points)

Write a function that rotates around the origin using forward mapping.

```
1 import numpy as np
2
3 def forward_mapping_origin(img, theta_deg):
4     # TODO: Implement forward mapping around origin
5     theta_rad = np.deg2rad(theta_deg)
6     src_h, src_w = img.shape[:2]
7     dst_h = src_h
8     dst_w = src_w
9
10    out = np.zeros((dst_h, dst_w, 3), dtype=np.uint8)
11
12    # YOUR CODE HERE
13    R = np.array([[np.cos(theta_rad), -np.sin(theta_rad)], [np.sin(theta_rad), np.cos(theta_rad)]])
14    y_indices, x_indices = np.indices((src_h, src_w))
15    x_indices = x_indices.ravel()
16    y_indices = y_indices.ravel()
17    coords = np.stack((x_indices, y_indices), axis=0)
18    rotated = np.matmul(R, coords)
19    xd, yd = np.round(rotated).astype(int)
20    mask = (xd >= 0) & (xd < dst_w) & (yd >= 0) & (yd < dst_h)
21    out[yd[mask], xd[mask]] = img[y_indices[mask], x_indices[mask]]
```

```
22
23     return out
```

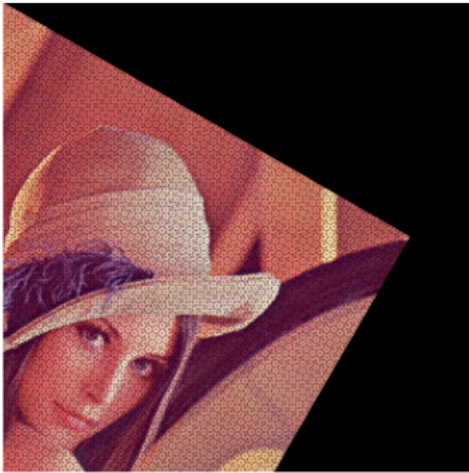
✓ Q1-2. Rotate by 30 degrees and display (5 points)

The image should look like it is rotated by -30 degree because in a Numpy array, the y-axis increases downward, whereas in Cartesian coordinates, the y-axis increases upward.

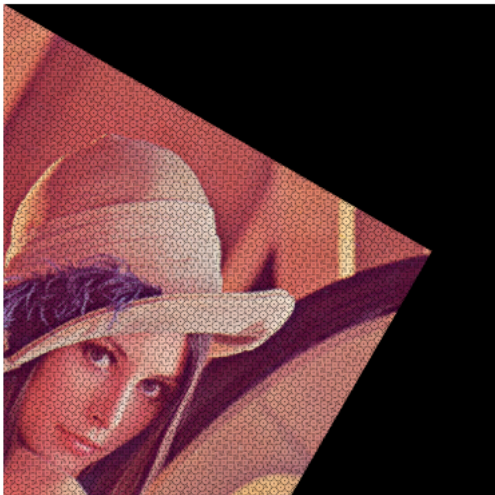
The expected output looks as follows:

```
1 expected_output = imageio.imread('HW0_Q1_2_expected_output.png')
2 plt.imshow(expected_output)
3 plt.axis('off')
4 plt.show()
```

```
/var/folders/hb/3k_51w_j5yd8v5cpqy1brfqr0000gn/T/ipykernel_46027/188146722.py:1: DeprecationWarning: Starting with Im
expected_output = imageio.imread('HW0_Q1_2_expected_output.png')
```



```
1 # Plot the output here.
2 rotated = forward_mapping_origin(img, 30)
3 plt.imshow(rotated)
4 plt.axis('off')
5 plt.show()
```



✓ Q1-3. Implement rotation around arbitrary coordinate (15 points)

Hint: shift coordinates so that the chosen point is treated as the origin before rotation.

```

1 def forward_mapping_arbitrary(img, theta_deg, cx, cy):
2     # TODO: Implement forward mapping around arbitrary point (cx, cy)
3     theta_rad = np.deg2rad(theta_deg)
4     src_h, src_w = img.shape[:2]
5     dst_h = src_h
6     dst_w = src_w
7
8     out = np.zeros((dst_h, dst_w, 3), dtype=np.uint8)
9
10    # YOUR CODE HERE
11    R = np.array([[np.cos(theta_rad), -np.sin(theta_rad)], [np.sin(theta_rad), np.cos(theta_rad)]])
12    y_indices, x_indices = np.indices((src_h, src_w))
13    x_indices = x_indices.ravel() - cx
14    y_indices = y_indices.ravel() - cy
15    coords = np.stack((x_indices, y_indices), axis=0)
16    rotated = np.matmul(R, coords)
17    xd, yd = np.round(rotated).astype(int)
18    xd += cx
19    yd += cy
20    mask = (xd >= 0) & (xd < dst_w) & (yd >= 0) & (yd < dst_h)
21    out[yd[mask], xd[mask]] = img[y_indices[mask] + cy, x_indices[mask] + cx]
22
23    return out

```

✓ Q1-4. Rotate with center (x=256, y=256), $\theta=30^\circ$ and display (5 points)

The expected output looks as follows:

```

1 expected_output = imageio.imread('HW0_Q1_4_expected_output.png')
2 plt.imshow(expected_output)
3 plt.axis('off')
4 plt.show()

```

/var/folders/hb/3k_51w_j5yd8v5cpqy1brfqr0000gn/T/ipykernel_46027/107794347.py:1: DeprecationWarning: Starting with ImageIO 2.19.0, the default image mode will be 'RGBA' instead of 'RGB'.
expected_output = imageio.imread('HW0_Q1_4_expected_output.png')



```

1 # Plot the output here.
2 rotated = forward_mapping_arbitrary(img, 30, 256, 256)
3 plt.imshow(rotated)
4 plt.axis('off')
5 plt.show()

```



Problems with forward mapping: holes (gap in the image) - they appear because some destination pixels are not assigned a value when multiple source pixels map to the same location or when no source pixel maps to a destination pixel. That is why we use **inverse mapping**.

✓ Part B. Inverse Mapping (30 points)

Explanation

- **Equation:**

$$x_s = R(-\theta) x_d$$

where

$$R(-\theta) = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix}$$

- **Insight:** Walk over each destination pixel and ask: where should I pull the color from in the source?
- **Pseudocode:**

```
for each pixel (x_d, y_d) in destination:
    [x_s, y_s] = R(-theta) * [x_d, y_d]
    destination(x_d, y_d) = sample_from_source(x_s, y_s)
```

✓ Q2-1. Implement `inverse_mapping_origin` (15 points)

```
1 def inverse_mapping_origin(img, theta_deg):
2     # TODO: Implement inverse mapping rotation around origin
3     theta_rad = np.deg2rad(theta_deg)
4     src_h, src_w = img.shape[:2]
5     dst_h = src_h
6     dst_w = src_w
7
8     out = np.zeros((dst_h, dst_w, 3), dtype=np.uint8)
9
10    # YOUR CODE HERE
11    R = np.array([[np.cos(-theta_rad), -np.sin(-theta_rad)], [np.sin(-theta_rad), np.cos(-theta_rad)]])
12    y_indices, x_indices = np.indices((dst_h, dst_w))
13    x_indices = x_indices.ravel()
14    y_indices = y_indices.ravel()
15    coords = np.stack((x_indices, y_indices), axis=0)
16    rotated = np.matmul(R, coords)
17    xs, ys = np.round(rotated).astype(int)
18    mask = (xs >= 0) & (xs < src_w) & (ys >= 0) & (ys < src_h)
19    out[y_indices[mask], x_indices[mask]] = img[ys[mask], xs[mask]]
20
21    return out
```

Q2-2. Display rotation ($\theta=30^\circ$) (5 points)

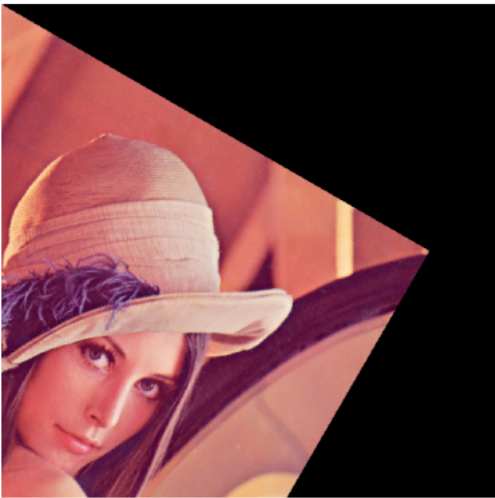
The expected output looks as follows:

```
1 expected_output = imageio.imread('HW0_Q2_2_expected_output.png')
2 plt.imshow(expected_output)
3 plt.axis('off')
4 plt.show()
```

/var/folders/hb/3k_51w_j5yd8v5cpqy1brfqr0000gn/T/ipykernel_46027/4232865078.py:1: DeprecationWarning: Starting with I
expected_output = imageio.imread('HW0_Q2_2_expected_output.png')



```
1 # Plot the output here.
2 rotated = inverse_mapping_origin(img, 30)
3 plt.imshow(rotated)
4 plt.axis('off')
5 plt.show()
```



Q2-3. Implement rotation around arbitrary center (15 points)

```
1 def inverse_mapping_arbitrary(img, theta_deg, cx, cy):
2     # TODO0: Implement inverse mapping around arbitrary point (cx, cy)
3     theta_rad = np.deg2rad(theta_deg)
4     src_h, src_w = img.shape[:2]
5     dst_h = src_h
6     dst_w = src_w
7
8     out = np.zeros((dst_h, dst_w, 3), dtype=np.uint8)
9
```

```

10 # YOUR CODE HERE
11 R = np.array([[np.cos(-theta_rad), -np.sin(-theta_rad)], [np.sin(-theta_rad), np.cos(-theta_rad)]])
12 y_indices, x_indices = np.indices((dst_h, dst_w))
13 x_indices = x_indices.ravel() - cx
14 y_indices = y_indices.ravel() - cy
15 coords = np.stack((x_indices, y_indices), axis=0)
16 rotated = np.matmul(R, coords)
17 xs, ys = np.round(rotated).astype(int)
18 xs += cx
19 ys += cy
20 mask = (xs >= 0) & (xs < src_w) & (ys >= 0) & (ys < src_h)
21 out[y_indices[mask] + cy, x_indices[mask] + cx] = img[ys[mask], xs[mask]]
22
23 return out

```

✓ Q2-4. Display result (x=256, y=256, $\theta=30^\circ$) (5 points)

The expected output looks as follows:

```

1 expected_output = imageio.imread('HW0_Q2_4_expected_output.png')
2 plt.imshow(expected_output)
3 plt.axis('off')
4 plt.show()

```

/var/folders/hb/3k_51w_j5yd8v5cpqy1brfqr0000gn/T/ipykernel_46027/431391595.py:1: DeprecationWarning: Starting with Im
expected_output = imageio.imread('HW0_Q2_4_expected_output.png')



```

1 # Plot the output here
2 rotated = inverse_mapping_arbitrary(img, 30, 256, 256)
3 plt.imshow(rotated)
4 plt.axis('off')
5 plt.show()

```



Part C. Animation (40 points)

Q3-1. Rotation animation (15 points)

Implement a function to rotate the image continuously around the center and save as an MP4.

```

1 import cv2
2
3 def make_rotation_video(img, seconds=3, fps=24, out_file='rotation.mp4'):
4     h, w = img.shape[:2]
5     center = (w//2, h//2)
6     fourcc = cv2.VideoWriter_fourcc(*'mp4v')
7     writer = cv2.VideoWriter(out_file, fourcc, fps, (w, h))
8
9     # YOUR CODE HERE
10    N = seconds * fps
11
12    for t in range(N):
13
14        theta = 360 * t / N
15
16        frame = inverse_mapping_arbitrary(img, theta, *center)
17        frame_bgr = cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)
18        writer.write(frame_bgr)
19    writer.release()
20
21
22 make_rotation_video(img, seconds=3, fps=24)
  
```

Q3-2. Turn Your Favorite Photo a Rotating Video (10 points)

```

1 # Load your own favorite photo
2 favorite_photo = imageio.imread('petergriffin.jpeg')
3 make_rotation_video(favorite_photo, seconds=3, fps=24, out_file='rotating_favorite_photo.mp4')
  
```

/var/folders/hb/3k_51w_j5yd8v5cpqy1brfqr0000gn/T/ipykernel_46027/1137990020.py:2: DeprecationWarning: Starting with I
 favorite_photo = imageio.imread('petergriffin.jpeg')

Q3-3. Creative Animation (15 points)

Create your own fun animation ('rotating_favorite_photo_creative.mp4'). For example:

- Image flying out of frame
- Zoom in/out while rotating
- Add trails or effects Be creative!

```
1 def inverse_zoom(img, scale_factor):
2     src_h, src_w = img.shape[:2]
3     dst_h = src_h
4     dst_w = src_w
5     cx = src_w // 2
6     cy = src_h // 2
7     out = np.zeros((dst_h, dst_w, 3), dtype=np.uint8)
8
9     yd, xd = np.indices((dst_h, dst_w))
10    yd = yd.ravel()
11    xd = xd.ravel()
12    coords = np.stack((xd - cx, yd - cy), axis=0)
13
14    scaled = coords / scale_factor
15    xs, ys = np.round(scaled).astype(int)
16    xs += cx
17    ys += cy
18
19    mask = (xs >= 0) & (xs < src_w) & (ys >= 0) & (ys < src_h)
20    out[yd[mask], xd[mask]] = img[ys[mask], xs[mask]]
21
22    return out
23
24 def shift(img, dx, dy):
25     src_h, src_w = img.shape[:2]
26     dst_h = src_h
27     dst_w = src_w
28     out = np.zeros((dst_h, dst_w, 3), dtype=np.uint8)
29
30     yd, xd = np.indices((dst_h, dst_w))
31     yd = yd.ravel()
32     xd = xd.ravel()
33     xs = xd - dx
34     ys = yd - dy
35
36     mask = (xs >= 0) & (xs < src_w) & (ys >= 0) & (ys < src_h)
37     out[yd[mask], xd[mask]] = img[ys[mask], xs[mask]]
38
39     return out
40
```